Собираем и подключаем

У вас есть план. Два файла из прошлого урока:

- **research.md** — что есть у конкурентов
- **brief.md** — какие экраны, что на каждом, куда ведут кнопки

Это ваш чертёж.
Сегодня мы этот чертёж превращаем в работающее приложение. Прямо сейчас. Без теории, без подготовки — у вас есть всё, что нужно.
Открывайте VS Code, запускайте Claude Code в папке проекта.

---

1. Создаём Mini App одним промптом

Вот промпт, который создаст ваше Mini App. Он длинный — и это хорошо. Чем подробнее описание, тем ближе результат к тому, что вы задумали.
Помните из Модуля 2: **хороший план = результат с первого раза, а не с десятого.**

```
Прочитай файлы в проекте:
research.md, brief.md.
На их основе создай Telegram Mini App.

Технические требования:
- HTML + CSS + JavaScript, без фреймворков
- Подключи Telegram Web App SDK
  (telegram-web-app.js) —
  для данных пользователя и темы Telegram
- Все файлы положи в папку tg-app/
- Точка входа — tg-app/index.html

Дизайн:
- Стиль нативного Telegram-приложения:
  закруглённые углы, мягкие тени,
  светлый фон, акцент (#2AABEE)
- Поддержка тёмной темы
- Выглядит как ПРИЛОЖЕНИЕ, не как сайт в рамке
- Плавные переходы между экранами
  (slide или fade, 200-300ms)

Мобильная адаптация:
- Только мобильный экран (320-430px)
- Кнопки минимум 44px в высоту
- Шрифт не менее 14px
- Нет горизонтальной прокрутки
- Касания отзываются визуально
```

```
Контент:
- Русский язык интерфейса
- Реальные данные из brief.md
  (не заглушки "Lorem ipsum")
- Если данных нет —
  придумай реалистичные для этой ниши

Дополнительно:
- Создай CLAUDE.md — какие файлы за что отвечают,
  навигация между экранами, где менять данные
- Комментарии на русском в каждом файле
- Задавай мне вопросы по ходу работы
```

Копировать

Не пугайтесь технических слов

CSS-переменные, active-состояния, SDK — это инструкция для Claude, не для вас. Просто скопируйте и отправьте. Claude разберётся.

Отправили? Claude начнёт работу. Через пару минут в папке `tg-app/` появятся файлы приложения. А в корне проекта — `CLAUDE.md` с описанием структуры.

Что происходит за кулисами

Claude читает ваши два файла — это как передать строителю чертёж и список пожеланий заказчика.

Он создаёт HTML-страницы (экраны), стилизует их под Telegram (CSS), добавляет переходы и интерактивность (JavaScript).

Файл `CLAUDE.md` — это карта вашего проекта.

Если через неделю вы захотите что-то поменять, не нужно разбираться в коде — откройте карту и скажите Claude: «Поменяй цены на экране услуг» или «Добавь новую категорию». Он найдёт нужный файл по карте.

---

2. Проверяем в браузере

Откройте `tg-app/index.html` в браузере. Самый простой способ — **перетащите файл** из папки прямо в окно браузера (Chrome, Safari — не важно).

Или скажите Claude: «Как открыть index.html в браузере?» — он подскажет для вашей системы.

Теперь важно: **проверяйте как на телефоне**, а не как на компьютере.

В Chrome: правый клик → «Просмотр кода» (Inspect) → иконка телефона в левом верхнем углу панели → выберите iPhone или Samsung. Экран сожмётся до мобильного размера.
Пролистайте все экраны. Нажмите на кнопки. Проверьте переходы.

---

3. Быстрая доработка

Что-то не нравится? **Один промпт = одно изменение.** Не пытайтесь исправить всё разом.
Примеры:

```
Сделай кнопки крупнее
и добавь отступ между ними.
Больше ничего не меняй.
```

Копировать

```
Замени фото-заглушки на иконки из emoji.
Не трогай остальные экраны.
```

Копировать

```
Экран [название вашего экрана] слишком пустой.
Добавь блок с отзывами — 3 коротких отзыва.
```

Копировать

Правило: один промпт = одна задача. Это не ограничение, а лайфхак. Маленькие точные правки дают предсказуемый результат. Большой запрос «поменяй всё» — путаницу и ошибки.

Если сломалось

Это нормально и легко исправить. Скажите: «Верни всё как было до последнего изменения.» Claude откатит за 5 секунд.

Что поняли

Один подробный промпт + два файла с планом = работающее приложение за пару минут.

Claude читает ваш чертёж и строит по нему. Чем точнее чертёж — тем меньше переделок.

А если что-то не так — маленькие правки по одной.

---

4. Приложение готово. Теперь — в Telegram

У вас есть работающее приложение в браузере. Но пока оно живёт только на вашем компьютере.
Чтобы оно заработало в Telegram — нужны два шага:

- Создать бота (это «дверь» в приложение)
- Сказать Telegram, где находится ваше приложение

Создаём бота через BotFather

BotFather — это специальный бот в Telegram, который создаёт других ботов. Считайте, что BotFather — это «паспортный стол» для ботов.

Шаг	Что делаем
1	Откройте Telegram → найдите @BotFather → Start
2	Напишите /newbot
3	BotFather спросит имя. Введите: «Каталог Анны» или «Beauty Master»
4	Потом спросит username — техническое имя, на `bot`: `anna_beauty_bot`
5	Готово. Скопируйте токен — длинную строку из букв и цифр

🔐 **Токен — это пароль от вашего бота**

Как ключ от квартиры. Нельзя показывать публично, отправлять в чат, выкладывать в интернет. Если кто-то получит ваш токен — сможет управлять ботом.

---

5. Сохраняем токен безопасно

Скопируйте токен и скажите Claude Code:

```
Создай файл .env для хранения токена бота.
Добавь .env в .gitignore,
чтобы токен не попал в публичный доступ.
Создай .env.example — шаблон без реальных значений,
чтобы было понятно какие переменные нужны.
Мой токен: [вставьте сюда свой токен]
```

Копировать

Claude создаст три файла. Давайте разберём каждый — они будут встречаться вам в каждом проекте.

Файл `.env` — сейф для секретов

Название расшифровывается как «environment» — «окружение». Это простой текстовый файл, в котором хранятся секретные данные проекта: токены, пароли, ключи от сервисов.

Внутри он выглядит просто:

```
BOT_TOKEN=7234567890:AAHfiqksKZ8tg9b3...
```

Копировать

Слева — название (чтобы вы и программа понимали, что это). Справа — значение (сам секрет).

Аналогия

`.env` — это сейф в вашем офисе. Внутри лежат ключи от всех замков: токен бота, пароль от базы данных, ключ от платёжной системы. Каждый проект, в котором есть хоть один секрет — должен иметь свой `.env`. Без исключений.

Этот файл **никогда** не отправляется в интернет, не выкладывается на GitHub, не пересылается в чат. Он живёт только на вашем компьютере (и на сервере, когда публикуете проект).

Файл `.env.example` — чертёж сейфа (без ключей)

Это копия `.env`, но без настоящих значений:

```
BOT_TOKEN=ваш_токен_сюда
```

Копировать

В чём разница?

- В `.env` лежит **настоящий** токен — тот, что выдал BotFather
- В `.env.example` — просто подсказка: «здесь должен быть токен»

Как витрина сейфа в магазине — показывает, какие полки внутри, но без ценностей.

Зачем нужен? Представьте: вы вернулись к проекту через полгода. Или передали его коллеге. Человек открывает папку и видит:

```
BOT_TOKEN=ваш_токен_сюда
DATABASE_URL=ваша_строка_подключения
```

Копировать

И сразу понимает: «Ага, нужно два ключа — токен бота и подключение к базе. Сейчас создам свой `.env` и вставлю свои значения.»

Без этого файла придётся гадать, какие секреты нужны проекту.

Этот файл безопасно хранить, показывать, публиковать — в нём нет настоящих секретов.

Файл `.gitignore` — охранник на проходной

Git — это система, которая запоминает все изменения в проекте (мы уже работали с ней в прошлых модулях).

Когда вы публикуете проект на GitHub — Git отправляет все файлы. **Все — кроме тех, что записаны в `.gitignore`.**

Аналогия

`.gitignore` — это список «не трогать». Как охранник на проходной офиса, у которого есть список: «Сейф — не выносить. Личные вещи — не выносить.»

Внутри файла — просто перечисление:

```
.env
node_modules/
```

Копировать

Git видит этот список и пропускает указанные файлы при публикации. Даже если вы случайно нажмёте «отправить всё» — секреты останутся на месте.

Итого — как они работают вместе

Файл	Что внутри	Публиковать?
`.env`	Настоящие токены и пароли	Нет, никогда
`.env.example`	Подсказка: какие ключи нужны	Да, безопасно
`.gitignore`	Список файлов, которые Git не отправляет	Да, нужно

Правило: в любом проекте, где есть токены, пароли или ключи — должны быть эти три файла. Claude Code знает это и создаст их автоматически. Но теперь вы понимаете, зачем они нужны.

Информация

Сейчас ваше приложение — простые HTML-файлы, и токен в коде не используется. Мы сохраняем его, чтобы не потерять и выработать привычку. Токен пригодится в уроке 12, когда подключим серверную логику бота.

---

6. Как передать токен Claude Code?

Есть два способа. Оба рабочие — выбирайте какой удобнее.

Способ 1. Вставить токен прямо в промпт — быстро и удобно
Вы пишете Claude Code: «Мой токен: 7234567890:AAH...» — он берёт его и записывает в `.env`.
Технически при этом токен проходит через серверы Anthropic (компания, которая создала Claude) — зашифрованный, защищённый, не публичный. Anthropic не хранит ваши данные и не использует их.
Для токена Telegram-бота это нормальный уровень безопасности — примерно как назвать номер карты оператору банка по телефону.

Способ 2. Вставить токен в файл вручную — максимальная безопасность
Попросите Claude Code создать файл `.env` с заглушкой:

```
Создай файл .env с переменной BOT_TOKEN=ВСТАВЬ_СЮДА.
Добавь .env в .gitignore. Создай .env.example.
```

Копировать

Затем откройте файл `.env` в VS Code (в меню слева, где видите все файлы — просто нажмите) и вручную замените `ВСТАВЬ_СЮДА` на настоящий токен.
Так токен вообще никуда не уходит — он попадает сразу в файл на вашем компьютере.

Совет

Для Telegram-бота — оба способа нормальные. Если бот скомпрометирован, токен легко перевыпустить: BotFather → /mybots → ваш бот → API Token → Revoke.

Привычка беречь секреты — полезная привычка.
Для серьёзных ключей (платёжные системы, базы данных с реальными клиентами) лучше использовать второй способ.

Где **нельзя** отправлять токены — никогда, ни при каких условиях:

- В публичные чаты и каналы Telegram
- В комментарии на GitHub (даже если потом удалите — история сохраняет всё)
- В сообщения на форумах и в соцсетях
- В код напрямую (вместо `.env`)

---

7. Публикуем приложение в интернет

Сейчас ваше приложение живёт только на вашем компьютере.
И вот ключевой момент: **Telegram не может его показать, пока файлы не окажутся в интернете.**

Почему?
Telegram Mini App — это веб-страница. Когда пользователь нажимает кнопку в боте, Telegram загружает страницу по адресу `https://...`. Если адреса нет — нечего загружать.

Аналогия

Это как написать красивое меню ресторана и оставить его дома — клиент у двери, а меню показать нечего.

Цепочка такая:

```
Файлы на компьютере
    ↓ публикуем в интернет
Ссылка https://my-app.vercel.app
    ↓ даём BotFather
Telegram загружает страницу
    ↓
Пользователь видит приложение
```

Копировать

Без публикации — цепочка рвётся. Это не опциональный шаг, а обязательный.

Помните Vercel из Модуля 2? Вы уже публиковали через него лендинг. Сейчас — то же самое:

```
Помоги опубликовать папку tg-app/
```

```
через Vercel.
Мне нужна HTTPS-ссылка для Mini App.

Важно: публиковать именно tg-app/,
а не весь проект.

Веди меня пошагово как новичка.
```

Копировать

Claude проведёт вас через процесс.

Root Directory = tg-app/

Если Vercel спросит про Root Directory — укажите **tg-app/**. Это папка, где лежит ваш index.html.

В итоге вы получите ссылку вида `https://название-проекта.vercel.app`. Telegram принимает **только HTTPS-ссылки** (с замочком) — и Vercel даёт их автоматически.

Проверьте перед следующим шагом: откройте эту ссылку в браузере телефона.
Видите своё приложение? Отлично, идём дальше.
Видите пустую страницу или ошибку? Деплой не удался. Скажите Claude:

```
Приложение не открывается по ссылке.
Помоги разобраться по шагам, я новичок.
От меня запрашивай только то,
что не можешь сделать сам.
```

Копировать

---

8. Подключаем приложение к боту

Теперь подключаем к боту. Здесь есть **два разных способа** — и они делают разные вещи.

Способ 1. Menu Button — кнопка внутри бота

Шаг	Что делаем

1	Вернитесь к **@BotFather**
2	**/mybots** → выберите бота
3	**Bot Settings** → **Menu Button** → **Configure**
4	Вставьте **ссылку Vercel**
5	Текст кнопки: [«Открыть каталог»]

Что это даёт: внизу чата с ботом, слева от поля ввода, появляется кнопка. Человек **уже зашёл в бот**, видит кнопку, нажимает — открывается ваше приложение. Это вход «изнутри» бота.

Способ 2. Mini App — прямая ссылка

Шаг	Что делаем
1	**/mybots** → бот → **Bot Settings** → **Mini Apps**
2	**Enable Mini Apps** → вставьте ту же ссылку

Что это даёт: появляется **прямая ссылка** вида `t.me/ваш_бот/app`.
Эту ссылку можно отправить кому угодно — в чат, в соцсети, на визитку.
Человек нажимает — и сразу попадает в приложение на полный экран.

Важный нюанс: даже через прямую ссылку Telegram передаёт приложению данные пользователя — имя и ID.
Ваше приложение «видит», кто его открыл, и может показывать персональный контент (например, «Привет, Анна!»). Человеку не нужно регистрироваться или логиниться — Telegram делает это автоматически.
Но есть разница: если человек зашёл через прямую ссылку, **не нажав /start в боте** — бот не сможет потом отправить ему сообщение. Ни подтверждение записи, ни напоминание, ни акцию.
Для этого человек должен хотя бы раз нажать /start — тогда бот «знает» о нём и может писать первым.

Аналогия

Представьте магазин в торговом центре. **Прямая ссылка** — адрес магазина на визитке. Человек заходит, смотрит витрину, может что-то выбрать. Но если он не оставил номер на входе (/start) — вы не позвоните с напоминанием. **Menu Button** — вывеска внутри ТЦ: чтобы её увидеть, нужно уже быть «внутри» (нажать /start), зато у вас сразу есть контакт.

Нужны оба? Да, настройте оба.

- Menu Button — для тех, кто уже в боте (у вас есть их контакт)
- Прямая ссылка — для привлечения новых людей (быстрый вход без обязательств)

Что поняли

Бот — это «дверь» в ваше приложение. BotFather создаёт эту дверь. Токен — ключ от неё, хранить в безопасности (.env). У приложения два входа: кнопка внутри бота (Menu Button) и прямая ссылка (Mini App) — настройте оба.

---

9. Лайфхак: экран-оффер для сбора контактов

Если человек зашёл через `t.me/бот/app` — он видит приложение, но бот о нём не знает. Чтобы это исправить, добавьте ненавязчивое предложение подписаться:

```
Добавь экран-оффер в Mini App.
Появляется при первом открытии,
показывать один раз
(запоминать в localStorage —
хранилище браузера).

Структура:

1. Эмодзи — крупная, по центру
   [🎁]

2. Заголовок — главная ценность
   [Скидка 15% на первую запись]

3. Подзаголовок — что нужно сделать
   [Подпишитесь на бота —
   получите промокод в личное сообщение]
```

```
4. Буллеты — выгоды подписки
   [• Напомним о записи за день]
   [• Первыми узнаёте о свободных окошках]
   [• Эксклюзивные акции для подписчиков]

5. Кнопка — яркая, на всю ширину
   Текст: [Получить скидку 15%]
   Ссылка: https://t.me/{username_бота}?start=from_app

6. Под кнопкой — «Пропустить»

Дизайн:
- Модалка с полупрозрачным фоном
- Карточка по центру, скруглённые углы
- Кнопка — акцент (#2AABEE), минимум 48px
- Заголовок — жирный, 20-24px
- Плавная анимация появления
```

Копировать

Внимание

Замените на username вашего бота (тот, что на `_bot`). Если отправите промпт как есть — ссылка не будет работать.

В квадратных скобках — пример для бьюти-мастера. **Замените на свою нишу:**

Ниша	Оффер (заголовок)	CTA (кнопка)
Бьюти-мастер	Скидка 15% на первую запись	Получить скидку 15%
Ресторан	Десерт в подарок к первому заказу	Получить подарок
Фотограф	Бесплатная мини-консультация	Записаться бесплатно
Репетитор	Первый урок — 0 ₽	Забрать бесплатный урок

Принцип: человек нажимает не на «подписаться», а на **«получить ценность»**. Подписка — побочный эффект, а не цель. Человек хочет скидку — и попутно становится подписчиком.

Что произойдёт: человек нажимает кнопку → открывается бот → срабатывает /start → бот теперь «знает» этого человека и может писать ему первым.
Параметр `?start=from_app` — это метка. Она говорит боту: «этот человек пришёл из приложения».
В уроке 12, когда добавим серверную логику, бот сможет по этой метке отправить специальное приветствие с бонусом вместо обычного «Привет». А пока — главное, что контакт сохраняется.

Информация

Это необязательный шаг. Приложение работает и без него. Но если хотите потом отправлять уведомления — это единственный способ получить «разрешение» на первое сообщение от бота. А оффер с ценностью даёт конверсию в 3–5 раз выше, чем просто «подпишитесь».

🚀 10. ВАУ-момент

Всё подключено. Бот создан, приложение в интернете, ссылки в BotFather.
Откройте Telegram. Найдите вашего бота. Нажмите кнопку.
Видите? Это приложение — **вы сделали**. Оно работает прямо в Telegram.
Любой человек может открыть ссылку `t.me/ваш_бот/app` — и увидит то, что вы создали.
Без скачивания, без регистрации, без App Store.
Вспомните, где вы были час назад: пустая папка и два файла с планом. Сейчас — работающее приложение в Telegram.
Это навык, а не привязка к Telegram.
То, что вы только что сделали — план → промпт → приложение → подключение к мессенджеру — работает в любом мессенджере. Принцип не меняется. Меняется только кнопка.

---

11. Три типа кнопок в боте

Вы настроили Menu Button — и она работает. В Telegram есть ещё два типа кнопок. Не обязательно использовать сейчас — просто знайте, что они существуют.

1. Menu Button — то, что вы уже сделали
Постоянная кнопка слева внизу, рядом с полем ввода. Всегда на виду. Один клик — и пользователь в приложении.
Когда использовать: у бота одна главная функция. Каталог, калькулятор, портфолио — нажал и открыл.

2. Inline-кнопки — кнопки под сообщением бота
Бот отправляет сообщение, а под ним — кнопки с вариантами. Например: «Выберите категорию: [Маникюр] [Брови] [Макияж]».
Пользователь нажимает — бот реагирует и в зависимости от нажатой кнопки сам присылает сообщения.
Вся эта логика заложилась на этапе формирования идеи и анализа конкурентов. Вот почему начальный этап такой важный.
Когда использовать: нужно предложить выбор. Квиз, навигация по разделам, подтверждение действия.

3. Reply Keyboard — большие кнопки вместо клавиатуры
Вместо обычной клавиатуры появляются крупные кнопки: [Каталог] [О нас] [Контакты]. Пользователь нажимает — бот получает текст.
Когда использовать: у бота несколько разделов, нужна постоянная навигация.

Все три типа можно **комбинировать** в одном боте:

- Menu Button открывает Mini App
- Inline-кнопки помогают с выбором внутри чата
- Reply Keyboard — для быстрой навигации

Вам не нужно прямо сейчас использовать все три. Просто знайте о возможностях. Когда понадобится — скажите Claude:

```
Добавь inline-кнопки для выбора категории
```

Копировать

Что поняли

Ваше приложение работает в Telegram. Три типа кнопок — для разных задач. Menu Button у вас уже есть. Остальные — по необходимости. Не усложняйте раньше времени.

---

12. Оформляем бота — один запрос

Когда кто-то находит вашего бота в Telegram — он видит описание. Это первое впечатление. Оно решает: нажмёт человек Start или пройдёт мимо.
Плохое описание: «Привет! Я бот.» Человек не понимает, зачем он здесь.
Хорошее описание отвечает на три вопроса:

- **Что** — что делает этот бот
- **Зачем** — какую пользу получит человек
- **Куда нажать** — чёткое следующее действие

Лайфхак: один запрос — и бот полностью оформлен
Claude Code знает ваш токен (он в файле `.env`). Через этот токен он может настроить бота напрямую — без ручной работы в BotFather.

```
Настрой моего Telegram-бота полностью.
Токен бота в файле .env.

Бот открывает Mini App —
[опиши в 1-2 предложениях
что делает приложение].

Через Telegram Bot API:
1. Установи описание бота
   (setMyDescription) —
   2-3 предложения: что, зачем, куда нажать
2. Установи короткое описание
   (setMyShortDescription) — одно предложение
3. Установи команды
   (setMyCommands) — /start, /help
   и как связаться с автором

В приложении (Mini App):
4. Экран приветствия (онбординг):
   - Обращение по имени (из Telegram)
   - Что здесь можно сделать (2-3 пункта)
   - Кнопка «Начать»
5. На каждом экране — понятные кнопки
6. Кнопку «Поделиться с другом» —
   отправляет ссылку на бота

Русский язык, дружелюбный тон, без пафоса.
```

Копировать

Claude сделает всё сам: зайдёт в Telegram через токен, настроит описание и команды бота, доработает приложение — добавит приветствие, кнопки, «Поделиться».
Вам ничего не нужно копировать и настраивать вручную.

---

13. Публикуем обновления

После всех доработок — один запрос:

```
Опубликуй проект.

Если нет репозитория на GitHub —
создай и свяжи с проектом.

Закоммить все изменения,
запуши на GitHub.

Задеплой обновлённую версию на Vercel.

Если Vercel уже подключён —
убедись, что пуш запустил деплой.
Делай всё сам, запрашивай у меня
только авторизацию если нужна.
```

Копировать

Claude сам проверит: есть ли репозиторий, настроен ли remote, всё ли закоммичено. Если чего-то нет — создаст и настроит.
Вам останется только открыть бота в Telegram и убедиться, что всё работает.

✅ 14. Финальная проверка

Если что-то не работает — не пытайтесь разбираться в коде. Опишите проблему Claude:

```
[Опишите что именно не работает].
Исправь. Я новичок.
```

Копировать

Совет

Чем конкретнее описание — тем быстрее результат. «Кнопка не работает» — плохо. «При нажатии на кнопку "Записаться" ничего не происходит» — хорошо.

Что поняли

Один запрос — и бот полностью оформлен: описание, команды, приветствие, кнопки, «Поделиться». Claude сам настраивает всё через токен. Вручную ничего делать не нужно.

📋 Домашнее задание

1. Доработайте приложение

Если вы работали вместе с уроком — у вас уже работающее приложение. Теперь улучшите его:

- Добавьте экран-оффер (промпт из раздела 9) — подставьте свою нишу и оффер
- Оформите бота одним запросом (промпт из раздела 12) — приветствие, кнопки, «Поделиться»
- Пройдите чек-лист ещё раз после доработок

Если не успели на уроке — догоняйте: промпт для создания Mini App и все шаги выше.

2. Проверьте на телефоне

Откройте бота на реальном телефоне (не в эмуляторе). Пройдите весь путь: /start → кнопка → листаете экраны → нажимаете кнопки. Запишите, что хотите улучшить.

3. Покажите кому-нибудь

Отправьте ссылку `t.me/ваш_бот/app` другу, коллеге, маме — так они сразу увидят приложение. Попросите открыть и пощупать. Два вопроса: «Что непонятно?» и «Где застрял?». Запишите ответы — пригодятся в следующем уроке.

Образ идеального результата

Вы открываете Telegram на телефоне. Находите своего бота. Нажимаете кнопку — открывается красивое приложение с вашими услугами, товарами или портфолио.
Всё на русском, кнопки крупные, переходы плавные. Вы отправляете ссылку другу — и он видит то же самое.
За один урок вы прошли путь от файлов с планом до работающего приложения в мессенджере. **Это ваше второе портфолио.**

Чек-лист

- Создал Mini App из файлов research.md и brief.md
- Проверил в браузере в мобильном режиме
- Создал бота через BotFather, сохранил токен в .env

Опубликовал на Vercel (получил HTTPS-ссылку)

Проверил ссылку — приложение загружается

Подключил к боту (Menu Button + Mini Apps)

Открыл бота в Telegram — работает

Оформил бота одним запросом (раздел 12)

(Бонус) Добавил экран-оффер

Что дальше

Сейчас ваше приложение — витрина. Кнопка «Записаться» пока не сохраняет данные, а бот пока не умеет отвечать на /start сам. Это нормально — в следующем уроке всё исправим:

- **Лайфхаки** — как дорабатывать приложение скриншотами с телефона и тестировать на живых людях
- **База данных** — подключим Supabase (бесплатный сервис — как онлайн-таблица), чтобы сохранять записи, заказы и контакты
- **Серверная логика бота** — научим бота отвечать на /start, отправлять промокоды и различать, откуда пришёл пользователь
- **Публикация** — опубликуем проект на VPS, чтобы бот работал 24/7

Но главное — у вас уже есть работающее приложение в Telegram. Всё остальное — доработка и усиление.

Готово

За один урок вы прошли путь от файлов с планом до работающего приложения в Telegram.

Любой человек может открыть вашего бота по ссылке — и увидеть то, что вы создали. 🎉

© Дмитрий Ледовских* Meta признана экстремистской организацией в РФ